

Describiendo un lenguaje: sintaxis

Laboratorio 02: sintaxis concreta y abstracta de MiniLisp++

M. en C.I.C. Manuel Soto Romero

Facultad de Ciencias, UNAM

Semestre 2027-1

Repaso mínimo

Ruta de análisis

En MINILISP₀₁ ya vimos la ruta:

$$\text{String} \xrightarrow{\text{ALEX}} [\text{Token}] \xrightarrow{\text{HAPPY}} \text{ASA}$$

Meta del laboratorio

Extender la sintaxis concreta y la sintaxis abstracta, sin definir todavía reglas de evaluación ni reglas de tipos.

MiniLisp++

EBNF de trabajo

```
 $\langle \text{Expr} \rangle ::= \langle \text{Num} \rangle \mid \langle \text{Bool} \rangle$   
          | (  $\langle \text{NaryOp} \rangle$   $\langle \text{Expr} \rangle$   $\langle \text{Expr} \rangle$  {  $\langle \text{Expr} \rangle$  } )  
          | (  $\langle \text{BinOp} \rangle$   $\langle \text{Expr} \rangle$   $\langle \text{Expr} \rangle$  )  
          | (  $\langle \text{UnOp} \rangle$   $\langle \text{Expr} \rangle$  )
```

Clasificación

NaryOp	and, or, +, -, *, /, <, >, <=, >=
BinOp	expt, eq
UnOp	not, add1, sub1, zero?

Corrección de diseño: `expt` y `eq` se tratan como operadores estrictamente binarios; las operaciones n-arias reciben al menos dos argumentos.

Ejemplo

Texto fuente

```
(and #t (or #f #t) (zero? 0))
```

Tokens esperados

```
[TokenPA, TokenAnd, TokenBool True,  
TokenPA, TokenOr, TokenBool False, TokenBool True, TokenPC,  
TokenPA, TokenZeroP, TokenNum 0, TokenPC,  
TokenPC]
```

ASA esperado

```
And [Boolean True,  
Or [Boolean False, Boolean True],  
ZeroP (Num 0)]
```

Reto 1: analizador léxico

Completa `Lexer.x`

Agrega las reglas de ALEX para reconocer:

- ▶ operadores lógicos: `and`, `or`;
- ▶ operadores aritméticos: `*`, `/`, `expt`;
- ▶ comparadores: `<`, `>`, `<=`, `>=`, `eq`;
- ▶ operadores unarios: `add1`, `sub1`, `zero?`.

Cuidado

`<=` y `>=` deben reconocerse como un solo *token*; no como `<` seguido de `=` o `>` seguido de `=`.

Reto 2: gramática

Completa `Grammars.y`

Agrega producciones para:

1. expresiones n-arias con al menos dos argumentos;
2. expresiones estrictamente binarias para `expt` y `eq`;
3. expresiones unarias para `not`, `add1`, `sub1`, `zero?`.

Forma esperada

```
(+ 1 2 3)           -- valido
(expt 2 3)          -- valido
(expt 2 3 4)        -- no valido
(zero? 0)           -- valido
(and #t)            -- no valido en este laboratorio
```

Reto 3: sintaxis abstracta

Acciones de construcción

Cada producción debe construir el ASA correspondiente. Por ejemplo:

(+ 1 2 3)	==> Add [Num 1, Num 2, Num 3]
(expt 2 3)	==> Expt (Num 2) (Num 3)
(zero? 0)	==> ZeroP (Num 0)

Argumenta tu decisión

En la defensa, explica por qué usaste listas para operadores n-arios y dos subárboles para `expt` y `eq`.

Reto 4: pruebas

Ejecuta el ciclo completo

```
alex Lexer.x  
happy Grammars.y  
runghc TestLaboratorio02.hs
```

Qué validan las pruebas

Las pruebas revisan *tokens*, ejemplos concretos, rechazo de entradas mal formadas y una propiedad de ida y vuelta:

ASA $\longrightarrow$ texto fuente $\longrightarrow$ tokens $\longrightarrow$ ASA.

Próxima semana: defensa en parejas

Entrega y ponderación

Expondrán en parejas cómo resolvieron cada reto en máximo 5 minutos. El script completo vale 40 % si pasa todas las pruebas; la defensa vale 60 %.

Rúbrica de la defensa	Puntos
Justifica la clasificación de operadores y las aridades elegidas.	15
Explica la relación texto fuente, <i>tokens</i> , gramática y ASA.	15
Defiende las acciones de HAPPY y el uso de listas en n-arios.	15
Analiza al menos un caso rechazado por las pruebas.	10
Distribuye el tiempo y la participación de la pareja con claridad.	5
Total defensa	60